

数值分析课程笔记

Tan Dongliang

Changchun University of Science and Technology

更新：2026 年 2 月 4 日

1 绪论

算法源码

1.1 学习数值分析的重要性

数值分析的核心是**算法构造**和**误差分析**。数值分析能根据近似的数据，采用近似的方法获得**满足要求**的近似结果。

怎样使计算机的计算结果可信和尽量减少出错的情况，在科学计算中是非常重要的，因为错误的计算结果会产生错误的结论或否定原来正确的数学模型。

计算机解决实际问题的四个步骤：

1. 建立数学模型
2. **选择数值方法**
3. 编写程序
4. 上机计算

1.2 计算机中的数系与运算

1.2.1 机器数系的定义

数学上的十进制浮点数可表示为：

$$x = \pm 10^c \times 0.a_1a_2\cdots, a_i \in \{0, 1, 2, \cdots, 9\}$$

类似的， β 进制的浮点数表示为：

$$x = \pm \beta^c \times 0.a_1a_2\cdots, a_i \in \{0, 1, 2, \cdots, \beta - 1\}$$

$$x = \pm \beta^c \times 0.a_1a_2\cdots a_t, a_i \in \{0, 1, 2, \cdots, \beta - 1\}$$

其中， t 为正整数，表示计算机的**字长**； c 为整数，表示**阶码**，满足 $L \leq c \leq U$ ， L 和 U 为固定整数。 $0.a_1a_2\cdots a_t$ 称为**尾数**。

注 当且仅当 $a_1 \neq 0$ 时, 称为规格化浮点数。

因此, 在计算机系统中, 实数系 $\mathbb{R}$ 是用所谓的浮点数系统 $\mathbb{F}$ (机器数系) 来近似表示的。

定义 1.1 (机器数系) 进制为 β 的浮点数系统表示为:

$$\mathbb{F}(\beta, t, L, U) = \{\pm\beta^c \times 0.a_1a_2\cdots a_t | a_i \in \{0, 1, 2, \dots, \beta-1\}, L \leq c \leq U\}$$

机器数系是有限的离散集, 其机器精度为 β^{-t} . 该数集整体在实数轴上的分布是不均匀的, 但相同阶数的数又等距分布在实数轴的某一段上。

机器数系中有绝对值最大和最小的非零数 M 和 m , 十进制浮点数系 $\mathbb{F}(10, 4, -99, 99)$, 绝对值最大的非零数 $M = 10^{99} \times 0.9999$, 绝对值最小的非零数 $m = 10^{-99} \times 0.0001$. 若数绝对值大于 M , 产生**上溢错误**, 小于 m , 则产生**下溢错误**. 上溢时, 中断程序; 下溢时, 用零表示该数继续程序; 无论是上溢, 还是下溢, 都称为**溢出错误**。

表1列出了 IEEE (Institute of Electrical and Electronics Engineers) 标准中规定的最常见的几种浮点数系统。

表 1: IEEE 浮点数系统

系统	进制 β	t	L	U
IEEE 单精度	2	24	-126	127
IEEE 双精度	2	53	-1022	1023
Cray	2	48	-16383	16384
HP calculator	2	12	-499	499
IBM mainframe	2	6	-64	63

1.2.2 机器数系的运算

计算机对输入数据 x (非零实数) 的处理采用以下方式:

1. 若 $x \in \mathbb{F}(\beta, t, L, U)$, 则原样接收 x
2. 若 $x \notin \mathbb{F}(\beta, t, L, U)$, 但 $m \leq |x| \leq M$, 则用 $\mathbb{F}(\beta, t, L, U)$ 中最接近 x 的数 $fl(x)$ 表示

机器数的运算规则如下:

- 加减法: **先对阶, 后运算, 再舍入**
- 乘除法: **先运算, 再舍入**

案例 1 某计算机数系 $\mathbb{F}(10, 4, -99, 99)$ 的两个数 $x_1 = 0.2337 \times 10^{-1}$ 和 $x_2 = 0.3364 \times 10^2$, 求这两个数相加、相乘的运算结果。

解: 相加运算

$$\begin{aligned}
 fl(x_1 + x_2) &= fl(0.2337 \times 10^{-1} + 0.3364 \times 10^2) \\
 &= fl(\underbrace{0.0002337 \times 10^2 + 0.3364 \times 10^2}_{\text{对阶}}) \\
 &= fl(0.3366337 \times 10^2) \\
 &= 0.3366 \times 10^2
 \end{aligned}$$

相乘运算

$$\begin{aligned} fl(x_1 \times x_2) &= fl(0.2337 \times 10^{-1} \times 0.3364 \times 10^2) \\ &= fl(\underbrace{0.7861667 \times 10^0}_{\text{直接运算}}) \\ &= fl(\underbrace{0.7862 \times 10^0}_{\text{舍入}}) \\ &= 0.7862 \end{aligned}$$

注 计算机的运算一般在中央处理器 (CPU) 中运行, 而 CPU 中的运算器一般是多倍字长存储, 因此, 参加运算的数据允许有超出原机器数系字长的数出现。计算结果需要保存在内存或硬盘, 因此需要严格按照机器数系的大小进行截断或舍入处理。

案例 2 在字长为 8 的十进制浮点数系统下, 给定 $x = 0.23371258 \times 10^{-4}$, $y = 0.33678429 \times 10^2$, $z = -0.33677811 \times 10^2$, 分别计算 $(x+y)+z$ 和 $x+(y+z)$ 的结果。

解:

$$\begin{aligned} (x+y)+z &= fl((0.23371258 \times 10^{-4} + 0.33678429 \times 10^2) - 0.33677811 \times 10^2) \\ &= fl(\underbrace{0.00000023371258 \times 10^2 + 0.33678429 \times 10^2 - 0.33677811 \times 10^2}_{\text{对阶}}) \\ &= fl(0.00000641371258 \times 10^2) \\ &= 0.641 \times 10^{-3} \end{aligned}$$

$$\begin{aligned} x+(y+z) &= fl(0.23371258 \times 10^{-4} + (0.33678429 \times 10^2 - 0.33677811 \times 10^2)) \\ &= fl(0.23371258 \times 10^{-4} + \underbrace{(0.33678429 \times 10^2 - 0.33677811 \times 10^2)}_{\text{对阶}}) \\ &= fl(0.23371258 \times 10^{-4} + 0.61800000 \times 10^{-3}) \\ &= fl(\underbrace{0.023371258 \times 10^{-3} + 0.61800000 \times 10^{-3}}_{\text{对阶}}) \\ &= fl(0.641371258 \times 10^{-3}) \\ &= 0.64137126 \times 10^{-3} \end{aligned}$$

而 $x+y+z$ 的精确值为 $0.641371258 \times 10^{-3}$ 。比较可以发现, $(x+y)+z \neq x+(y+z)$, 而且 $x+(y+z)$ 要比 $(x+y)+z$ 精确的多。这是由于计算机上实行对位及有限字长导致的“大数吃小数”现象。因此, 设计算法时, 要尽可能避免一个很大的数和一个很小的数相加的问题。多个数相加时, 应从绝对值较小的数依次加起, 以避免有效数字的损失。

1.3 误差

1.3.1 误差的定义

精确值与近似值的差异就是误差。

定义 1.2 (绝对误差) 设 x 是准确值, x^* 是 x 的一个近似值, 称差 $x^* - x$ 为近似值 x^* 的绝对误差, 简称误差, 记为 e^* 或 $e(x^*)$, 即 $e(x^*) = x^* - x$.

定义 1.3 (绝对误差限) 称满足 $|e^*| = |x^* - x| \leq \varepsilon^*$ 的正数 ε^* 或 $\varepsilon(x^*)$ 为近似值 x^* 的绝对误差限.

定义 1.4 (相对误差) 设 x 是准确值, x^* 是 x 的一个近似值, 称 $\frac{e^*}{x} = \frac{x^* - x}{x}$ 为近似值 x^* 的相对误差, 记为 e_r^* 或 $e_r(x^*)$, 即

$$e_r(x^*) = \frac{e^*}{x} = \frac{x^* - x}{x}$$

相对误差的绝对值越小, 近似程度越高。

定义 1.5 (相对误差限) 称满足 $|e_r^*| = \left| \frac{x^* - x}{x} \right| \leq \varepsilon_r^*$ 的正数 ε_r^* 或 $\varepsilon_r(x^*)$ 为近似值 x^* 的相对误差限.

1.3.2 数值计算的误差

定理 1.1 (四则运算的误差) 假设 x^* 和 y^* 分别是准确值 x 和 y 的一个近似值, 则有

1. 四则运算的绝对误差估计:

$$\begin{aligned} e(x^* \pm y^*) &= e(x^*) \pm e(y^*) \\ e(x^* y^*) &\approx y^* e(x^*) + x^* e(y^*) \\ e\left(\frac{x^*}{y^*}\right) &\approx \frac{y^* e(x^*) - x^* e(y^*)}{(y^*)^2} \end{aligned}$$

2. 四则运算的相对误差估计:

$$\begin{aligned} e_r(x^* \pm y^*) &\approx \frac{x^* e_r(x^*) \pm y^* e_r(y^*)}{x^* \pm y^*} \\ e_r(x^* y^*) &\approx e_r(x^*) + e_r(y^*) \\ e_r\left(\frac{x^*}{y^*}\right) &\approx e_r(x^*) - e_r(y^*) \end{aligned}$$

定理 1.2 (多元函数的误差估计) 设多元函数 $u = f(x_1, x_2, \dots, x_n)$, 自变量 $(x_1, x_2, \dots, x_n)$ 的近似值为 $(x_1^*, x_2^*, \dots, x_n^*)$, 则有多元函数 $f(x_1, x_2, \dots, x_n)$ 的误差估计

$$\begin{aligned} (1) \quad e(f(x_1^*, x_2^*, \dots, x_n^*)) &\approx \sum_{i=1}^n \frac{\partial f(x_1^*, x_2^*, \dots, x_n^*)}{\partial x_i} e(x_i^*) \\ (2) \quad \varepsilon(f(x_1^*, x_2^*, \dots, x_n^*)) &\approx \sum_{i=1}^n \left| \frac{\partial f(x_1^*, x_2^*, \dots, x_n^*)}{\partial x_i} \right| \varepsilon(x_i^*) \\ (3) \quad \varepsilon_r(f(x_1^*, x_2^*, \dots, x_n^*)) &\approx \sum_{i=1}^n \left| \frac{\partial f(x_1^*, x_2^*, \dots, x_n^*)}{\partial x_i} \right| \frac{\varepsilon(x_i^*)}{|f(x_1^*, x_2^*, \dots, x_n^*)|} \end{aligned}$$

1.3.3 计算机的计算误差

设计算机的数系为 $\mathbb{F}(\beta, t, L, U)$, m 及 M 是其中绝对值最小及最大的正数, 某数 $x = \pm \beta^c \times 0.a_1 a_2 \dots, a_1 \neq 0$, 满足 $m < |x| < M, x \notin \mathbb{F}(\beta, t, L, U)$, 则计算机经舍入处理后以数 $fl(x)$ 接收, 即

$$fl(x) = \pm \beta^c \times \tilde{a}, \tilde{a} = \begin{cases} 0.a_1 a_2 \dots a_t & , \quad 0 \leq a_{t+1} < \beta/2 \\ 0.a_1 a_2 \dots a_t + \beta^{-t}, & a_{t+1} \geq \beta/2 \end{cases}$$

因此计算机对 x 的计算绝对误差和计算相对误差有如下估计

$$(1) |e(\text{fl}(x))| = |x - \text{fl}(x)| \leq 0.5 \times \beta^{c-t}$$

$$(2) |e_r(\text{fl}(x))| = \frac{|x - \text{fl}(x)|}{|x|} \leq \frac{0.5 \times \beta^{c-t}}{0.1 \times \beta^c} = 0.5 \times \beta^{1-t}$$

由此可知，计算机对任何实数的计算相对误差限与实数本身无关，只与计算机字长 t 有关，其值为 $0.5 \times \beta^{1-t}$ 。因此通常定义数 $\text{eps} = 0.5 \times \beta^{1-t}$ 为计算机的精度。

1.4 有效数字

1.5 数值分析中的常用概念

数值分析的研究内容有如下两个方面：

- 连续系统的离散化，如求连续函数的定积分
- 离散型方程的数值求解，主要体现在怎样将来自实际问题的各种微分方程转化为合适的离散型方程，以便能通过该离散化方程来求出原来微分方程的近似解

考察数值分析中的求解公式，会看到其中大部分内容都是在这两大方面所做的研究得出的。一个算法所需要的乘法和除法总次数称为计算量，常用 N 表示。计算量的单位为 flop，表示完成一次浮点数乘法或除法所需要的时间。初始数据的微小变化，导致计算结果的剧烈变化的问题称为病态问题。初始数据的微小变化只引起计算结果的微小变化的计算问题称为良态问题。

如果一个算法进行计算的初始数据有误差，而在计算过程中产生的误差不增长，则称该算法为数值稳定算法，否则称为数值不稳定算法。

科学计算中值得注意的四个方面：

- 避免相近二数相减
- 避免用接近零的数做除数
- 控制误差的积累和传播，避免大数吃小数，用数值稳定的算法
- 简化计算过程，化简公式或想办法减少运算次数

2 非线性方程的求根方法

一般情况下，绝大部分非线性方程没有求根公式。数值分析中所采用的求根方法一般分为区间法（如二分法）和迭代法（如简单迭代法、牛顿迭代法等）两类，其共同点都是构造收敛于根的数列 $\{x_k\}$ ，不同的是产生 $\{x_k\}$ 的方法不一样。

定义 2.1 (非线性方程) 设 $f(x)$ 为一元连续函数，称方程 $f(x) = 0$ 为函数方程；当 $f(x)$ 不是 x 的线性函数时，称对应的函数方程为非线性方程。

衡量一种求根方法的优劣，可由其产生的数列 $\{x_k\}$ 收敛于根 x^* 的快慢决定，为此引入收敛阶的概念。

定义 2.2 (收敛阶) 设数列 $\{x_k\}$ 收敛于 x^* ，若存在正数 p 和 C ，满足

$$\lim_{k \rightarrow \infty} \frac{|x_{k+1} - x^*|}{|x_k - x^*|^p} = C$$

则称 $\{x_k\}$ 的收敛阶为 p 或 $\{x_k\}$ 收敛于 x^* 的速度是 p 阶的；同时称产生 $\{x_k\}$ 的方法具有 p 阶敛速。

注 本节所讨论的求根方法一次只能求一个根，对求方程多个根的问题，可利用函数 $f(x)$ 的特点先将根进行隔离，使每个区间只含有一个根，再利用本节求根方法分别求解。

2.1 二分法

基本思想：利用连续函数零点定理，将含根区间逐次减半缩小，构造出收敛点列来逼近根。

2.1.1 构造原理

微积分中的零点定理指出：若 $f(x) \in C[a, b]$ ，且满足 $f(a)f(b) < 0$ ，则在区间 (a, b) 中至少有一点 ξ ，使得 $f(\xi) = 0$ 。将含根区间分为两个长度相等的子区间后，在这两个子区间上也可利用零点定理确定根在哪个子区间上，在这一系列子区间中依次选择一个点就可以构造出数列 $\{x_k\}$ ，由于含根区间是以逐次减半的方式减小并最后趋于零，因此数列 $\{x_k\}$ 会收敛于根 x^* 。

二分法构造过程

- (1) 记含根区间 $I_0 = [a, b]$ ，取 I_0 中点 $x_0 = 0.5(a + b)$ ，并计算 $f(x_0)$ ；
- (2) 判别 $f(x_0)$ 的值，
 - 若 $f(x_0) = 0$ ，则 $x^* = x_0$ ，终止；
 - 若 $f(a)f(x_0) < 0$ ，则 $x^* \in [a, x_0]$ ，取 $I_1 = [a, x_0]$ ；
 - 若 $f(a)f(x_0) > 0$ ，则 $x^* \in [x_0, b]$ ，取 $I_1 = [x_0, b]$ ；
- (3) 记 $I_1 = [a_1, b_1]$ ，取 I_1 中点 $x_1 = 0.5(a_1 + b_1)$ ，
若 x_1 满足根的精度要求，取 $x^* \approx x_1$ ，终止；否则用 I_1 代替 I_0 ，转 (1)。

2.1.2 收敛性

因为根 $x^* \in I_k$ ($k = 0, 1, \dots$)，且 $I_0 \supset I_1 \supset I_2 \supset \dots$ ，所以有 x_k 的误差满足

$$|x^* - x_k| \leq \frac{1}{2}(b_k - a_k) = \frac{1}{2^2}(b_{k-1} - a_{k-1}) = \dots = \frac{1}{2^{k+1}}(b - a).$$

于是当 $k \rightarrow \infty$ 时，由 $\frac{1}{2^{k+1}}(b - a) \rightarrow 0$ 得到 $x_k \rightarrow x^*$ ，即二分法产生的数列 $\{x_k\}$ 收敛于根 x^* 。给定精度 $\epsilon > 0$ 后，可以使用 $K = \lceil \frac{\ln(b-a) - \ln \epsilon}{\ln 2} - 1 \rceil$ 或 $|x_k - x_{k-1}| \leq \epsilon$ 作为终止条件，此时， x_k 是满足精度的根，证明略。

2.2 简单迭代法

简单迭代法是非线性方程求根方法中各类迭代法的基础，也可以很方便的扩展应用到方程（组）求解问题中。

基本思想：将方程 $f(x) = 0$ 等价变形为 $x = \varphi(x)$ ，获得迭代公式 $x_{k+1} = \varphi(x_k)$ ，再由它计算出逼近根 x^* 的数列 $\{x_k\}$ 。

2.2.1 构造原理

若 x^* 为根, 则有 $f(x^*) = 0$, 故 $x^* = \varphi(x^*)$ 成立, 说明根 x^* 可以经过函数 $\varphi(x)$ 计算出来。

证明. 当 $\varphi(x)$ 产生的迭代数列 $\{x_k\}$ 收敛时, 一定收敛到根 x^* . 设 $\lim_{k \rightarrow \infty} x_k = \bar{x}$, 且迭代函数 $\varphi(x)$ 一般是连续的, 利用连续函数与极限的关系, 有

$$\bar{x} = \lim_{k \rightarrow \infty} x_{k+1} = \lim_{k \rightarrow \infty} \varphi(x_k) = \varphi(\lim_{k \rightarrow \infty} x_k) = \varphi(\bar{x}),$$

这说明迭代数列 $\{x_k\}$ 的极限就是所求的根, 即 $\bar{x} = x^*$.

简单迭代法构造过程

- (1) 将方程 $f(x) = 0$ 改写为等价形式 $x = \varphi(x)$;
- (2) 构造迭代公式 $x_{k+1} = \varphi(x_k)$;
- (3) 取初值 x_0 , 由迭代公式计算数列 $x_1 = \varphi(x_0), x_2 = \varphi(x_1), \dots$.

数列 $\{x_k\}$ 称为迭代数列, $\varphi(x)$ 为迭代函数。对于根 x^* , $x^* = \varphi(x^*)$, 即 $\varphi(x)$ 变不动它, 这样的点 (x^*) 形象地称为 $\varphi(x)$ 的不动点, 方程 $x = \varphi(x)$ 为不动点方程。因此, 有时也说非线性方程求根问题就是求对应的迭代函数的不动点问题。

实际中经常使用的是加速的 Aitken 迭代法, 这里直接给出算法流程:

Aitken 迭代法

Require: 初值 x^0 , 根的精度 ϵ , 迭代函数 $\varphi(x)$

Ensure: 方程的根 x^*

- 1: $x_k \leftarrow x_0$
- 2: 计算 $x_{k+1}^{(1)} = \varphi(x_k)$, $x_{k+1}^{(2)} = \varphi(x_{k+1}^{(1)})$
- 3: 计算 $x_{k+1} = x_k - \frac{(x_{k+1}^{(1)} - x_k)^2}{x_{k+1}^{(2)} - 2x_{k+1}^{(1)} + x_k}$
- 4: 判定 x_{k+1} 是否满足精度 ϵ , 满足则输出根 $x^* = x_{k+1}$; 否则, $x_k \leftarrow x_{k+1}$, 返回步骤 2.

2.2.2 收敛性

简单迭代法通常是线性收敛的, 个别情况下可以达到超线性收敛或更高阶收敛。以下定理证明, 当迭代函数 $\varphi(x)$ 满足一定条件时, 其产生的迭代数列是收敛的, 迭代公式可以收敛到根 x^* .

定理 2.1 (Lipschitz continuity) 设迭代函数 $\varphi(x)$ 满足下列两个条件:

- (1) 当 $x \in [a, b]$ 时, 有 $\varphi(x) \in [a, b]$;
- (2) 任取 $x_1, x_2 \in [a, b]$, 存在与 x_1 和 x_2 无关的正常数 $L < 1$, 满足

$$|\varphi(x_1) - \varphi(x_2)| \leq L|x_1 - x_2| \quad (1)$$

则 $\varphi(x)$ 在 $[a, b]$ 中有唯一不动点 x^* , 且迭代公式 $x_{k+1} = \varphi(x_k)$ 对任取 $x_0 \in [a, b]$, 产生的数列 $\{x_k\}$ 都收敛于 x^* .

证明. 作辅助函数 $\psi(x) = x - \varphi(x)$. **(存在性)** 由条件 (1) 知 $\psi(a)\psi(b) \leq 0$. 由零点定理, 存在 $\xi \in [a, b]$, 使 $\psi(\xi) = 0$, 由此得出 $\xi = \varphi(\xi)$, 说明 $\varphi(x)$ 在 $[a, b]$ 上有不动点 ξ . **(唯一性)** 假设存在另一不动点 η , 有 $\eta = \varphi(\eta)$, 利用条件 (2) 可得

$$|\xi - \eta| = |\varphi(\xi) - \varphi(\eta)| \leq L|\xi - \eta| < |\xi - \eta|,$$

等式前后矛盾, 因此满足定理条件下, $\varphi(x)$ 在 $[a, b]$ 有唯一不动点。**(收敛性)** 利用 x^* 是不动点、 x_k 的迭代公式及条件 (2), 有

$$|x_k - x^*| = |\varphi(x_{k-1}) - \varphi(x^*)| \leq L|x_{k-1} - x^*| \leq L^2|x_{k-2} - x^*| \leq L^k|x_0 - x^*|,$$

注意到 $0 < L < 1$, 因此, 当 $k \rightarrow \infty$ 时, 可得 $L^k \rightarrow 0$, 因此有

$$\lim_{k \rightarrow \infty} |x_k - x^*| = 0$$

故 $\lim_{k \rightarrow \infty} x_k = x^*$.

定理 2.2 (推论) 设迭代函数 $\varphi(x)$ 满足:

- (1) 当 $x \in [a, b]$ 时, 有 $\varphi(x) \in [a, b]$;
- (2) 存在正常数 $L < 1$, 满足 $|\varphi'(x)| \leq L$

则 $\varphi(x)$ 在 $[a, b]$ 中有唯一不动点 x^* , 对任取 $x_0 \in [a, b]$, 产生的数列 $\{x_k\}$ 都收敛于 x^* . 证明略。

2.2.3 误差估计

在给定精度 $\epsilon > 0$ 后, 要使 $|x^* - x_k| < \epsilon$, 只要计算到 $\frac{L}{1-L}|x_k - x_{k-1}| < \epsilon$ 或 $\frac{L^k}{1-L}|x_1 - x_0| < \epsilon$ 即可。也可以直接求出满足要求的迭代次数:

$$K = \left\lceil \ln \left| \frac{(1-L)\epsilon}{x_1 - x_0} \right| / \ln L \right\rceil.$$

实际中都用 $|x_k - x_{k-1}| \leq \epsilon$ 作为终止条件, 通常也能求出满足精度要求的根。

证明略。

2.3 Newton 迭代法

牛顿迭代法 (Newton's method) 又称切线法或牛顿-拉夫逊方法 (Newton-Raphson method), 是牛顿在研究求根问题时借助大胆近似技术得出的目前求根方法中收敛最快的方法。

基本思想: 将函数 $f(x)$ 做线性化处理 (将 $f(x)$ 在某点展开为 Taylor 公式后, 取其线性部分 $L(x)$ 代替 $f(x)$), 把方程 $f(x) = 0$ 转化为对应的近似方程 $L(x) = 0$, 再从 $L(x) = 0$ 中构造迭代公式。

牛顿迭代法具有超线性收敛速度, 但实际上它一般具有平方收敛速度。

2.3.1 构造原理

Newton 迭代法通过不断求解原函数的近似函数方程来构造迭代数列, 直至收敛。

Newton 迭代法构造过程

(1) 将方程 $f(x)$ 在 x_k 处做 Taylor 展开

$$f(x) = f(x_k) + f'(x_k)(x - x_k) + \frac{f''(x_k)}{2!}(x - x_k)^2 + \dots$$

(2) 取一阶线性部分 $L(x) = f(x_k) + f'(x_k)(x - x_k)$, 求解近似方程 $L(x) = 0$, 有

$$f(x_k) + f'(x_k)(x - x_k) = 0$$

(3) 从步骤 (2) 中求得解, 记为 x_{k+1} , 得到迭代关系式

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \quad (2)$$

式(2)称为 Newton 迭代公式, 用此公式求方程 $f(x) = 0$ 根的方法称为 Newton 迭代法。

2.3.2 收敛性

实际上 Newton 迭代法需要选择一个尽可能靠近 x^* 的初值才能保证收敛。

以下定理给出了牛顿迭代法收敛的充分条件:

定理 2.3 (Newton 迭代法收敛的充分条件) 设函数 $f(x) \in C^2[a, b]$, 满足下列三个条件:

- (1) $f(a)f(b) < 0$;
- (2) $f'(x) \neq 0, x \in (a, b)$;
- (3) $f''(x), x \in (a, b)$ 存在且不变号。

则在 $[a, b]$ 内任取一点 x_0 , 只要 $f(x_0)f''(x_0) > 0$, Newton 迭代法产生的数列一定收敛于 $[a, b]$ 上的唯一根 x^* 。

2.3.3 Newton 迭代法的变形与推广

Newton 迭代法的变形: 割线法 为了克服 Newton 迭代法需要求导数 $f'(x)$ 的不便, 采用 $f'(x)$ 的近似值

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$$

将其代入牛顿迭代公式(2)可得割线法的迭代公式

$$x_{k+1} = x_k - \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} f(x_k).$$

割线法需要两个初值才能产生迭代数列, 因此是多点(两点)迭代公式。割线法是超线性收敛的, 收敛阶为 1.618。

Newton 迭代法的推广: 求解非线性方程组 变元数为 n 的非线性方程组的向量形式为

$$F(x) = 0$$

其中,

$$\mathbf{F}(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ \vdots \\ f_n(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} f_1(x_1, x_2, \dots, x_n) \\ f_2(x_1, x_2, \dots, x_n) \\ \vdots \\ f_n(x_1, x_2, \dots, x_n) \end{bmatrix}, \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad (3)$$

非线性方程组的 Newton 迭代公式为

$$\mathbf{x}_{k+1} = \mathbf{x}_k - (\nabla_{\mathbf{x}} \mathbf{F})^{-1} \mathbf{F}(\mathbf{x}_k)$$

3 线性方程组的解法

3.1 基本概念

3.1.1 线性方程组的概念与表达形式

线性方程组的一般形式为

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m \end{cases} \quad (4)$$

其中, a_{ij} 称为系数, b_i 称为右端项。其矩阵表示为 $\mathbf{Ax} = \mathbf{b}$, 式中

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix}, \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

当 $m < n$ 时, 通常方程组有无穷组解; 当 $m > n$ 时, 方程组可能无解; 只有当 $m = n$ 且系数矩阵 $\mathbf{A}$ 可逆时才有唯一解。本节主要讨论 $m = n$ 的线性方程组的求解问题, 且假设它有唯一解。

注 线性方程组的数值解法主要分为直接法和迭代法两大类。直接法是可以直接通过数学公式直接计算出准确解的方法, 通常适用于中小型规模的线性方程组求解。迭代法通过构造迭代公式, 然后以一个猜测的向量作为初始向量逐步迭代计算, 来获得满足精度要求的近似解。通常适用于大规模线性方程组的求解, 特别是大型稀疏线性方程组。

3.1.2 范数的定义

数列 $\{x_k\}$ 收敛于极限 a 的本质是随着 k 的增大, 数列点 x_k 与 a 的距离趋于零。因此, 距离是描述收敛的本质。对于向量序列, 其收敛计算的定义如下:

定义 3.1 设向量序列 $\mathbf{x}^{(k)} = (x_1^{(k)}, x_2^{(k)}, \dots, x_n^{(k)})^T$, 以及向量 $\mathbf{x}^* = (x_1^*, x_2^*, \dots, x_n^*)^T$, 如果有 $\lim_{k \rightarrow \infty} x_i^{(k)} = x_i^*$ 成立, 则称 $\mathbf{x}^{(k)}$ 收敛于 $\mathbf{x}^*$, 简记为 $\lim_{k \rightarrow \infty} \mathbf{x}^{(k)} = \mathbf{x}^*$ 。

注 向量序列中, 使用上标表示迭代索引, 下表表示向量分量。

实用中常用向量差的“范数”描述向量空间中元素之间的距离，其定义如下：

定义 3.2 (范数) 设 L 是数域 K 上的一个线性空间，如果定义在 L 上的实值函数 $P(x)$ 满足

(1) 任意 $x \in L$, 有 $P(x) \geq 0$, 且 $P(x) = 0 \Leftrightarrow x = 0$;

(2) 任意 $x \in L, \lambda \in K$, 有 $P(\lambda x) = |\lambda|P(x)$;

(3) 任意 $x, y \in L$, 有 $P(x + y) \leq P(x) + P(y)$,

则称 $P(\cdot)$ 是 L 上的一个范数，称 $P(x)$ 为 x 的一个范数。

范数的定义很像绝对值函数，故常用 $\|\cdot\|_P$ 或 $\|\cdot\|$ 表示范数，而范数 $P(x)$ 常记为 $\|x\|_P$ 或 $\|x\|$. 这样，上面的范数定义三个条件常写为：

范数的定义：

(1) 任意 $x \in L$, 有 $\|x\| \geq 0$, 且 $\|x\| = 0 \Leftrightarrow x = 0$;

(2) 任意 $x \in L, \lambda \in K$, 有 $\|\lambda x\| = |\lambda| \cdot \|x\|$;

(3) 任意 $x, y \in L$, 有 $\|x + y\| \leq \|x\| + \|y\|$.

注 定义了范数的线性空间称为赋范线性空间。

数值分析中常用的线性空间有以下几种：

- (1) 向量空间 (n 维): $\mathbb{R}^n = \{\mathbf{a} | \mathbf{a} = (a_1, a_2, \dots, a_n), a_k \in \mathbb{R}\}$
- (2) 矩阵空间: $\mathbb{R}^{m \times n} = \{\mathbf{A}_{m \times n} | \mathbf{A}_{m \times n} = (a_{ij})_{m \times n}, a_{ij} \in \mathbb{R}\}$
- (3) 连续函数空间: $C[a, b] = \{f(x) | f(x) \text{ 在 } [a, b] \text{ 上连续}\}$

数值分析中常用的范数有以下几种：

向量 ($\mathbb{R}^n$) 范数

- $\|x\|_1 = \sum_{k=1}^n |x_k|$
- $\|x\|_2 = \sqrt{\sum_{k=1}^n x_k^2}$
- $\|x\|_\infty = \max_{1 \leq k \leq n} |x_k|$

矩阵范数

- 列范数: $\|\mathbf{A}\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|$
- 行范数: $\|\mathbf{A}\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}|$
- F 范数: $\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^n a_{ij}^2}$
- 2 范数: $\|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}}, \lambda_{\max}$ 是 $\mathbf{A}^T \mathbf{A}$

注 矩阵范数需要额外满足 $\|AB\| \leq \|A\| \cdot \|B\|, A, B \in \mathbb{R}^{n \times n}$

定理 3.1 $\mathbb{R}^n$ 上的所有范数都是等价的。证明略。

定理 3.2 $\lim_{k \rightarrow \infty} \mathbf{x}^{(k)} = \mathbf{x}^* \Leftrightarrow \|\mathbf{x}^{(k)} - \mathbf{x}^*\| = 0$. 证明略。

判断向量数列收敛，只需要找到一种范数满足 $\lim_{k \rightarrow \infty} \|\mathbf{x}^{(k)} - \mathbf{x}^*\| = 0$ 即可。

3.1.3 谱半径与范数

谱半径是矩阵所有特征值的绝对值中最大的那一个。定义如下：

定义 3.3 (谱半径) 设 $\mathbf{A} \in R^{n \times n}$, $\lambda_k (k = 1, 2, \dots, n)$ 是 $\mathbf{A}$ 的 n 个特征值，则称实数

$$\rho(\mathbf{A}) = \max_{1 \leq k \leq n} |\lambda_k|$$

为矩阵 $\mathbf{A}$ 的谱半径。注意如果 λ_k 是复数， $|\lambda_k|$ 表示复数模。

定理 3.3 设 $\mathbf{A} \in R^{n \times n}$ ，则有 $\rho(\mathbf{A}) \leq \|\mathbf{A}\|$ ， $\|\mathbf{A}\|$ 为矩阵的任意矩阵范数。

证明. 设 λ_k 为 $\mathbf{A}$ 的任意特征值， $\mathbf{x}^{(k)}$ 为对应的特征向量，则有 $\mathbf{A}\mathbf{x}^{(k)} = \lambda_k \mathbf{x}^{(k)}$ ，取范数得

$$\|\mathbf{A}\| \|\mathbf{x}^{(k)}\| \geq \|\mathbf{A}\mathbf{x}^{(k)}\| = \|\lambda_k \mathbf{x}^{(k)}\| = |\lambda_k| \cdot \|\mathbf{x}^{(k)}\|,$$

由于 $\|\mathbf{x}^{(k)}\|$ 的任意性，可得

$$|\lambda_k| \leq \|\mathbf{A}\|.$$

得证.

定理 3.4 设 $\mathbf{A} \in R^{n \times n}$ ，则 $\lim_{k \rightarrow \infty} \mathbf{A}^k = 0 \Leftrightarrow \rho(\mathbf{A}) < 1$. 证明略.

3.2 线性方程组的迭代解法

线性方程组的迭代解法主要有 Jacobi 迭代法、Seidel(Guass-Seidel) 迭代法及 Sor(Successive over relaxation) 法等。

基本思想: 将线性方程组 $\mathbf{Ax} = \mathbf{b}$ 等价变形为 $\mathbf{x} = \mathbf{Bx} + \mathbf{g}$ 以构造向量迭代公式 $\mathbf{x}^{(k+1)} = \mathbf{Bx}^{(k)} + \mathbf{g}$ ，用计算出的向量序列 $\{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots\}$ 逼近准确解。

3.2.1 构造原理

Jacobi 迭代法 类比非线性方程求根的简单迭代法，Jacobi 迭代法的构造方法如 Jacobi 迭代法中所示。其简写形式为：

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right), i = 1, 2, \dots, n \quad (6)$$

下面引入 Jacobi 迭代法的矩阵形式。将公式5消除分母可得：

$$\begin{cases} a_{11}x_1^{(k+1)} = b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)} - \dots - a_{1n}x_n^{(k)} \\ a_{22}x_2^{(k+1)} = b_2 - a_{21}x_1^{(k)} - a_{23}x_3^{(k)} - \dots - a_{2n}x_n^{(k)} \\ \vdots \\ a_{nn}x_n^{(k+1)} = b_n - a_{n1}x_1^{(k)} - a_{n2}x_2^{(k)} - \dots - a_{n,n-1}x_{n-1}^{(k)} \end{cases} \quad (7)$$

Jacobi 迭代法构造过程

(1) 将第 i 行方程的第 i 个变量 x_i 用其他 $n-1$ 个变量表示:

$$\begin{cases} x_1 = \frac{1}{a_{11}} (b_1 - a_{12}x_2 - a_{13}x_3 - \cdots - a_{1n}x_n) \\ x_2 = \frac{1}{a_{22}} (b_2 - a_{21}x_1 - a_{23}x_3 - \cdots - a_{2n}x_n) \\ \vdots \\ x_n = \frac{1}{a_{nn}} (b_n - a_{n1}x_1 - a_{n2}x_2 - \cdots - a_{n,n-1}x_{n-1}) \end{cases} \quad (5)$$

(2) 根据公式5构造迭代公式

$$\begin{cases} x_1^{(k+1)} = \frac{1}{a_{11}} (b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)} - \cdots - a_{1n}x_n^{(k)}) \\ x_2^{(k+1)} = \frac{1}{a_{22}} (b_2 - a_{21}x_1^{(k)} - a_{23}x_3^{(k)} - \cdots - a_{2n}x_n^{(k)}) \\ \vdots \\ x_n^{(k+1)} = \frac{1}{a_{nn}} (b_n - a_{n1}x_1^{(k)} - a_{n2}x_2^{(k)} - \cdots - a_{n,n-1}x_{n-1}^{(k)}) \end{cases}$$

(3) 取初值 $\mathbf{x}^{(0)}$, 由迭代公式计算数列 $\{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots\}$, 直至收敛.

公式7的矩阵形式为:

$$\begin{bmatrix} a_{11} & & & \\ & a_{22} & & \\ & & \ddots & \\ & & & a_{nn} \end{bmatrix} \begin{bmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \\ \vdots \\ x_n^{(k+1)} \end{bmatrix} = \begin{bmatrix} 0 & -a_{12} & -a_{13} & \cdots & -a_{1n} \\ -a_{21} & 0 & -a_{23} & \cdots & -a_{2n} \\ \vdots & \ddots & \ddots & & \vdots \\ -a_{n1} & -a_{n2} & \cdots & -a_{n,n-1} & 0 \end{bmatrix} \begin{bmatrix} x_1^{(k)} \\ x_2^{(k)} \\ \vdots \\ x_n^{(k)} \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

记

$$D = \begin{bmatrix} a_{11} & & & \\ & a_{22} & & \\ & & \ddots & \\ & & & a_{nn} \end{bmatrix}, L = \begin{bmatrix} 0 & 0 & \cdots & 0 \\ -a_{21} & 0 & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ -a_{n1} & -a_{n2} & \cdots & 0 \end{bmatrix}$$

$$U = \begin{bmatrix} 0 & -a_{12} & \cdots & -a_{1n} \\ 0 & 0 & \cdots & -a_{2n} \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}, x^{(k)} = \begin{bmatrix} x_1^{(k)} \\ x_2^{(k)} \\ \vdots \\ x_n^{(k)} \end{bmatrix}, b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

则有 $A = D - L - U$, $D\mathbf{x}^{(k+1)} = (L + U)\mathbf{x}^{(k)} + \mathbf{b}$. 因为 $|D| \neq 0$, 矩阵 D 可逆, 因此得到 Jacobi 迭代的向量公式:

$$\mathbf{x}^{(k+1)} = D^{-1}(L + U)\mathbf{x}^{(k)} + D^{-1}\mathbf{b} \quad (8)$$

令 $B_J = D^{-1}(L + U)$, $\mathbf{g}_J = D^{-1}\mathbf{b}$, 则迭代公式8可简写为:

$$\mathbf{x}^{(k+1)} = B_J\mathbf{x}^{(k)} + \mathbf{g}_J \quad (9)$$

Seidel 迭代法 在一次迭代过程中, 及时的用迭代计算出的新分量 $x_i^{(k+1)}$ 取代 $x_i^{(k)}$ 参加后面分量的计算, 就得到了如下的 Seidel 迭代公式:

$$\begin{aligned} x_1^{(k+1)} &= \frac{1}{a_{11}} \left(b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)} - \cdots - a_{1n}x_n^{(k)} \right) \\ x_2^{(k+1)} &= \frac{1}{a_{22}} \left(b_2 - a_{21}x_1^{(k+1)} - a_{23}x_3^{(k)} - \cdots - a_{2n}x_n^{(k)} \right) \\ &\vdots \\ x_n^{(k+1)} &= \frac{1}{a_{nn}} \left(b_n - a_{n1}x_1^{(k+1)} - a_{n2}x_2^{(k+1)} - \cdots - a_{n,n-1}x_{n-1}^{(k+1)} \right) \end{aligned}$$

简写形式为:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right), i = 1, 2, \dots, n \quad (10)$$

当 Seidel 和 Jacobi 都能收敛的情况下, Seidel 能收敛的更快。类似 Jacobi 迭代的向量格式推导, Seidel 迭代有如下的矩阵形式:

$$D\mathbf{x}^{(k+1)} = L\mathbf{x}^{(k+1)} + U\mathbf{x}^{(k)} + \mathbf{b}$$

解出 $\mathbf{x}^{(k+1)}$, 可得 Seidel 的向量迭代公式:

$$\mathbf{x}^{(k+1)} = B_S\mathbf{x}^{(k)} + \mathbf{g}_S$$

其中, $B_S = (D - L)^{-1}U$, $\mathbf{g}_S = (D - L)^{-1}\mathbf{b}$. 矩阵 B_S 称为 Seidel 迭代矩阵。

Sor 迭代法 Sor 法具有提高收敛速度的功能, 其主要思路为在 Seidel 迭代的基础上, 根据当前迭代下 Seidel 计算的结果 $\mathbf{x}_S^{(k+1)}$ 与 $\mathbf{x}^{(k)}$ 相减得到差向量 $\Delta\mathbf{x}$, 再用参数 ω 乘之加到 $\mathbf{x}^{(k)}$ 上作为 $\mathbf{x}^{(k)}$ 的下一步迭代 $\mathbf{x}^{(k+1)}$. 其简写形式为:

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right)$$

参数 ω 为松弛因子, 可以任意选取。当 $\omega = 1$ 时, Sor 法就是 Seidel 迭代法。其向量形式为:

$$\mathbf{x}^{(k+1)} = B_\omega\mathbf{x}^{(k)} + \mathbf{g}_\omega$$

其中, $B_\omega = (D - \omega L)^{-1}[(1 - \omega)D + \omega U]$, $\mathbf{g}_\omega = \omega(D - \omega L)^{-1}\mathbf{b}$. 矩阵 B_ω 称为超松弛迭代矩阵。

3.2.2 收敛条件与误差估计

比较 Jacobi、seidel、Sor 可知, 这三种迭代公式可以用一个统一的迭代格式

$$\mathbf{x}^{(k+1)} = B\mathbf{x}^{(k)} + \mathbf{g} \quad (11)$$

表示。迭代矩阵 B 是常数矩阵, 故称之为单步定常线性迭代公式。其收敛条件有以下定理:

定理 3.5 单步定常线性迭代公式 $\mathbf{x}^{(k+1)} = B\mathbf{x}^{(k)} + \mathbf{g}$ 对任意初始向量 $\mathbf{x}^{(0)}$ 都收敛的充要条件是迭代矩阵谱半径 $\rho(B) < 1$.

证明. 必要性 设 $\lim_{k \rightarrow \infty} \mathbf{x}^{(k)} = \mathbf{x}^*$, 在 $\mathbf{x}^{(k+1)} = B\mathbf{x}^{(k)} + \mathbf{g}$ 中令 $k \rightarrow \infty$, 得 $\mathbf{x}^* = B\mathbf{x}^* + \mathbf{g}$, 两式相减并把 $k+1$ 记为 k , 得

$$\mathbf{x}^{(k)} - \mathbf{x}^* = B(\mathbf{x}^{(k-1)} - \mathbf{x}^*) = B^2(\mathbf{x}^{(k-2)} - \mathbf{x}^*) = \cdots = B^k(\mathbf{x}^{(0)} - \mathbf{x}^*)$$

由 $\lim_{k \rightarrow \infty} \mathbf{x}^{(k)} = \mathbf{x}^*$ 及 $\mathbf{x}^{(0)}$ 得任意性, 有 $\lim_{k \rightarrow \infty} \mathbf{B}^k = 0$. 再由定理3.4可得 $\rho(\mathbf{B}) < 1$.

充分性 因为 $\rho(\mathbf{B}) < 1$, 则有 $\mathbf{I} - \mathbf{B}$ 非奇异 ($\mathbf{I}$ 为单位阵), 从而线性方程组 $(\mathbf{I} - \mathbf{B})\mathbf{x} = \mathbf{g}$ 有唯一解 $\mathbf{x}^*$, 即成立 $(\mathbf{I} - \mathbf{B})\mathbf{x}^* = \mathbf{g}$, 于是有 $\mathbf{x}^* = \mathbf{B}\mathbf{x}^* + \mathbf{g}$, 类似必要性处理, 有

$$\mathbf{x}^{(k)} - \mathbf{x}^* = \mathbf{B}(\mathbf{x}^{(k-1)} - \mathbf{x}^*) = \mathbf{B}^2(\mathbf{x}^{(k-2)} - \mathbf{x}^*) = \cdots = \mathbf{B}^k(\mathbf{x}^{(0)} - \mathbf{x}^*) \quad (12)$$

由定理3.4, 有 $\lim_{k \rightarrow \infty} \mathbf{B}^k = 0$, 式12取极限, 可得 $\lim_{k \rightarrow \infty} \mathbf{x}^{(k)} = \mathbf{x}^*$.

得证.

由于谱半径 $\rho(\mathbf{B}) < 1$ 一般不易计算, 因此这个定理通常只用在理论上. 以下不加证明的给出三个迭代收敛的充分条件:

- (1) $\|\mathbf{B}\| < 1$, $\|\mathbf{B}\|$ 是 $\mathbf{B}$ 的任意矩阵范数
- (2) 若 $\mathbf{A}$ 是严格对角占优矩阵, 则 $\mathbf{A}\mathbf{x} = \mathbf{b}$ 的 Jacobi 和 Seidel 迭代对任意初始向量都收敛
- (3) 若 $\mathbf{A}$ 是对称正定矩阵, 则 $\mathbf{A}\mathbf{x} = \mathbf{b}$ 的 Seidel 迭代对任意初始向量都收敛

误差估计 设迭代矩阵 $\mathbf{B}$ 的某种范数 $\|\mathbf{B}\| < 1$, 有如下误差估计:

(1) 事后估计式:

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\| \leq \frac{\|\mathbf{B}\|}{1 - \|\mathbf{B}\|} \|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|$$

(2) 事前估计式:

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\| \leq \frac{\|\mathbf{B}\|^k}{1 - \|\mathbf{B}\|} \|\mathbf{x}^{(1)} - \mathbf{x}^{(0)}\|$$

3.3 线性方程组的直接解法

略.

3.4 解对系数的敏感性

方程中由于系数的变化 (扰动) 导致所求解的变化称为**解对系数的敏感性**. 计算机中由于计算误差的存在, 构成了对线性方程组系数矩阵和常数项的扰动, 导致求解出的解可能失真. 通常称**解对系数敏感的方程组为病态方程组**; 否则称为良态方程组.

设方程组 $\mathbf{A}\mathbf{x} = \mathbf{b}$ 经扰动后的方程组为: $(\mathbf{A} + \delta\mathbf{A})\mathbf{x} = \mathbf{b} + \delta\mathbf{b}$, 其准确解分别为 $\mathbf{x}^*$ 和 $\mathbf{x}^* + \delta\mathbf{x}$, 于是有等式:

$$\mathbf{A}\mathbf{x}^* = \mathbf{b}, \quad (\mathbf{A} + \delta\mathbf{A})(\mathbf{x}^* + \delta\mathbf{x}) = \mathbf{b} + \delta\mathbf{b}$$

解对系数的敏感性可以通过 $\mathbf{x}^* + \delta\mathbf{x}$ 和 $\mathbf{x}^*$ 的相对误差限来度量. 通过范数不等式, 有以下不等关系:

(1) $\delta \mathbf{A} = \mathbf{0}, \delta \mathbf{b} \neq \mathbf{0}$:

$$\frac{\|\delta \mathbf{x}\|}{\|\mathbf{x}^*\|} \leq \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\| \cdot \frac{\|\delta \mathbf{b}\|}{\|\mathbf{b}\|}$$

(2) $\delta \mathbf{A} \neq \mathbf{0}, \delta \mathbf{b} = \mathbf{0}$:

$$\frac{\|\delta \mathbf{x}\|}{\|\mathbf{x}^*\|} \leq \frac{\|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\| \cdot \frac{\|\delta \mathbf{A}\|}{\|\mathbf{A}\|}}{1 - \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\| \cdot \frac{\|\delta \mathbf{A}\|}{\|\mathbf{A}\|}}$$

(3) $\delta \mathbf{A} \neq \mathbf{0}, \delta \mathbf{b} \neq \mathbf{0}$:

$$\frac{\|\delta \mathbf{x}\|}{\|\mathbf{x}^*\|} \leq \frac{\|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\|}{1 - \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\| \cdot \frac{\|\delta \mathbf{A}\|}{\|\mathbf{A}\|}} \left(\frac{\|\delta \mathbf{A}\|}{\|\mathbf{A}\|} + \frac{\|\delta \mathbf{b}\|}{\|\mathbf{b}\|} \right)$$

根据以上不等式，可以看出相对误差都与 $\|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\|$ 有关，且其大小直接影响相对误差限。因此，给出以下定义：

定义 3.4 设非奇异矩阵 $\mathbf{A} \in \mathbf{R}^{n \times n}$ ，称

$$\text{Cond}_p(\mathbf{A}) = \|\mathbf{A}\|_p \cdot \|\mathbf{A}^{-1}\|_p$$

为矩阵 $\mathbf{A}$ 的条件数，式中 $\|\cdot\|_p$ 是矩阵范数。

实际中，由于计算 $\mathbf{A}^{-1}$ 相对困难和复杂，因此很少直接计算矩阵条件数来判断方程组是否病态，而是通过以下病态矩阵的经验特征进行观察：

- 行列式的某些行和列近似相关
- 元素间相差大数量级，且无规则
- 主元消去过程中出现小主元
- 特征值相差大数量级

科学计算中，常用残向量 $\mathbf{r} = \mathbf{A}\bar{\mathbf{x}} - \mathbf{b}$ 的无穷范数 $\|\mathbf{r}\|_\infty$ 来检验所求出的解 $\bar{\mathbf{x}}$ 是否准确。需要注意的是，该结论只在方程组是良态条件下才成立，以下给出简要证明：

证明. 设 $\mathbf{x}^*$ 为准确解， $\bar{\mathbf{x}} = \mathbf{x}^* + \delta \mathbf{x}$ 为计算的解，由 $\mathbf{r} = \mathbf{A}\bar{\mathbf{x}} - \mathbf{b}$ 可得

$$\mathbf{A}(\mathbf{x}^* + \delta \mathbf{x}) = \mathbf{A}\bar{\mathbf{x}} = \mathbf{b} + \mathbf{r}$$

将 $\mathbf{r}$ 看作 $\delta \mathbf{b}$ ，由上述误差关系式可得：

$$\frac{\|\bar{\mathbf{x}} - \mathbf{x}^*\|}{\|\mathbf{x}^*\|} \leq \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\| \cdot \frac{\|\mathbf{r}\|}{\|\mathbf{b}\|} = \text{Cond}(\mathbf{A}) \frac{\|\mathbf{r}\|}{\|\mathbf{b}\|}. \quad (13)$$

由式13可以看出，当 $\|\mathbf{r}\|$ 很小，但 $\text{Cond}(\mathbf{A})$ 很大，即方程组为病态时，计算解 $\bar{\mathbf{x}}$ 与准确解 $\mathbf{x}^*$ 的相对误差也可能很大，也就是说，病态方程组无法通过残向量的范数大小来判断结果的准确性。